RO03 - Jeux, graphes et RO

Zacharie ALES, Dylan LAPLACE-MERMOUD

2025 - 2026

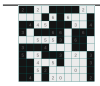
PROJET

1 Objectif

L'objectif de ce projet est de résoudre des jeux de logique issus du site :

chiark.greenend.org.uk/~sgtatham/puzzles/.

Vous choisirez deux des problèmes représentés dans le tableau suivant (la somme de leurs difficultés devra être au moins égale à 10).

	Nom	Difficulté	Objectif
	Mosaic	1	Place des cases noires en respectant les contraintes.
	Unequal	2	Remplir en respectant les inégalités.
	Pattern	3	Placer les cases selon le motif.
	Lightup	3	Placer des lampes pour éclairer l'ensemble des cases.
	Tents	3	Placer une tente pour chaque arbre.
	Unruly	3	Trouver un pavage de tuiles noires et blanches.
	Flip	3	Rendre toutes les cases blanches.
	Tracks	4	Relier par des rails.
	Dominosa	4	Trouver un pavage en dominos.
	Towers	4	Remplir par des tours.
	Undead	4	Remplir par des monstres.
	Rect	4	Trouver un pavage de rectangles.

	Nom	Difficulté	Objectif
	Signpost	4	Numéroter toutes les cases.
	Bridges	4	Associer le bon nombre de ponts aux sommets.
	Singles	4	Masquer les doublons des lignes et colonnes.
	Range	5	Placer les cases noires.
	Loopy	6	Trouver une boucle vérifiant les contraintes d'adjacence.
	Pearl	7	Trouver une boucle passant par chaque perle.
	Palisade	8	Placer des palissades vérifiant des contraintes d'adjacence.
	Filling	8	Remplir par des chiffres.
	Galaxies	9	Placer des barrières symétriquement.

Remarque 1 : Ces jeux sont également accessibles sur android (peut être également sur iOS).
Remarque 2 : Le programme *wine* peut permettre d'exécuter les fichiers .exe sous linux.

2 Travail demandé

Le travail demandé pour chacun de vos deux problèmes est décrit ci-dessous.

Remarque : Le code complet du jeu de sudoku est donné en guise d'exemple. N'hésitez pas à vous en inspirer pour chacun de vos deux jeux.

Partie 1 Implémentation

1. **Modélisation du problème**
   Modéliser sur papier la résolution du jeu par un programme linéaire en nombres entiers.

   Remarque : Le but de la plupart de ces problèmes est de trouver une solution réalisable. Il n'y a donc généralement pas d'objectif à proprement parlé. Ainsi, vous pouvez fixer une valeur d'objectif constante.

   Remarque 2 : Les problèmes de difficulté ≥ 6 contiennent chacun une règle qu'il est difficile de prendre en compte dans un programme linéaire en nombres entiers. En effet, cette règle nécessite d'utiliser un nombre de contraintes dans le modèle qui est exponentiel par rapport à la taille des données. Si vous n'arrivez pas à modéliser complètement cette règle, essayez au moins de le faire partiellement (*i.e.,* de

réduire le nombre de solutions pouvant poser problème). Pour prendre en compte ces contraintes, nous utiliserons un *callback* (voir exemple détaillé appliqué au sudoku en Section 4).

2. **Définition manuelle d'une instance**
   Dans le répertoire `data` créer un fichier `instanceTest.txt` représentant une instance simple du jeu considéré. Par exemple, dans le cas du sudoku, on pourrait créer le fichier suivant représentant une grille de taille 4×4 :

   Case initialement vide

   Première ligne de la grille → 1, , , 4

   , 3, , 1

   , , , 3

   La première case de la dernière ligne contient un 3 → 3, , ,

3. **Lecture d'une instance**
   a) Compléter la méthode `readInputFile(path::String)`, figurant dans le fichier `io.jl`, permettant la lecture d'une instance du jeu considéré figurant au chemin `path` et retournant une ou plusieurs variables représentant cette instance.
   b) Tester votre méthode en lançant julia en mode console, puis en exécutant la commande :
   `readInputFile("./data/instanceTest.txt")`.

4. **Implémentation du modèle**
   a) Implémenter votre modèle dans la méthode `cplexSolve`, figurant dans le fichier `resolution.jl`. Cette méthode retournera :
   — un booléen indiquant si une solution optimale est obtenue ;
   — un réel indiquant la durée de la résolution ;
   — une ou plusieurs variables représentant la solution.
   b) Afin de prendre en compte la règle difficile de votre jeu de difficulté ≥ 6, utiliser un callback (voir exemple détaillé appliqué au sudoku en Section 4) permettant au cours de la résolution :
   — de tester chaque solution trouvée par CPLEX ;
   — d'ajouter une contrainte excluant la solution si elle ne vérifie pas la règle.
   c) Tester la résolution de l'instance figurant dans votre fichier test en lançant julia en mode console puis en exécutant les commandes :
   ```
   include("resolution.jl")
   cplexSolve(readInputFile("../data/test.txt"))
   ```
   Remarque : Pour que cela fonctionne, les entrées de la méthode `cplexSolve` doivent correspondre aux sorties de la méthode `readInputFile`.

5. **Affichage en ligne de commande**
   Créer deux méthodes `displayGrid()` et `displaySolution()` dans le fichier `io.jl` permettant respectivement d'afficher dans la console :
   — une instance non résolue ;
   — une instance résolue.
   Exemple pour une grille de sudoku :

```
1,  ,  ,  4
 ,  3,  ,  1
 ,  ,  ,  3
3,  ,  ,
```

Instance au format texte

```
-------------
| 1  -  | -  4 |
| -  3  | -  1 |
-------------
| -  -  | -  3 |
| 3  -  | -  - |
-------------
```

`displayGrid()`

```
-------------
| 1  2  | 3  4 |
| 4  3  | 2  1 |
-------------
| 2  1  | 4  3 |
| 3  4  | 1  2 |
-------------
```

`displaySolution()`

6. **Génération d'instances**
   a) Compléter la méthode `generateInstance()` figurant dans le fichier `generation.jl` afin qu'elle permette de générer une instance aléatoire du jeu considéré. Vous pourrez ajouter des paramètres à cette méthode afin de générer différents types d'instances. Par exemple dans le cas du sudoku, la taille de la grille et le pourcentage de cases remplies sont considérés.
   b) Compléter la méthode `generateDataSet()` du fichier `generation.jl` afin de générer un ensemble d'instances dans le répertoire `./data`.

   Remarque : Il n'est pas nécessaire que les instances générées possèdent une unique solution.
7. **Résolution de plusieurs instances**
   a) Compléter la méthode `solveDataSet()` afin qu'elle permette d'effectuer la résolution de l'ensemble des instances.

   Remarque : Les résultats de cplex pour chaque instance seront stockés dans un fichier texte figurant dans le répertoire `./res/cplex`. Chaque fichier devra contenir deux variables `solveTime` et `isOptimal`. Ces conditions sont nécessaires au bon fonctionnement des méthodes de visualisation des résultats que nous utiliserons par la suite.
   b) *(à effectuer uniquement pour votre jeu le plus difficile)* Compléter la méthode `heuristicSolve()` du fichier `resolution.jl` afin qu'elle réalise une résolution heuristique d'une instance. Votre heuristique peut être simple mais ne doit néanmoins pas uniquement consister à énumérer l'ensemble des solutions.
8. **Mise en forme des résultats**
   Utiliser la méthode `resultsArray()` du fichier `io.jl` afin de représenter vos résultats sous forme de tableaux.

Partie 2 Rapport (date limite : 8 mai)

Le rapport contiendra la description de vos programmes linéaires, vos heuristiques, vos générateurs d'instances et la façon dont vous avez implémenter le callback pour votre problème de difficulté ≥ 6. Vous commenterez les résultats de vos méthodes de résolution :

— temps de calcul en fonction de la taille ou du type d'instances (graphiques encouragés) ;
— une solution est-elle toujours trouvée ?
— jusqu'à quelle taille la résolution est-elle possible ?
— si le programme linéaire est trop long et que vous avez une fonction objectif qui n'est pas constante, quel est le gap après 10 minutes ?

  Définition - *Gap*

  Ecart relatif entre la meilleure relaxation linéaire r et la meilleure solution entière connue z : $\frac{|r-z|}{z}$;
— ...

Vous fournirez également l'ensemble du code source de votre projet.

Partie 3 Soutenance (11 mai)

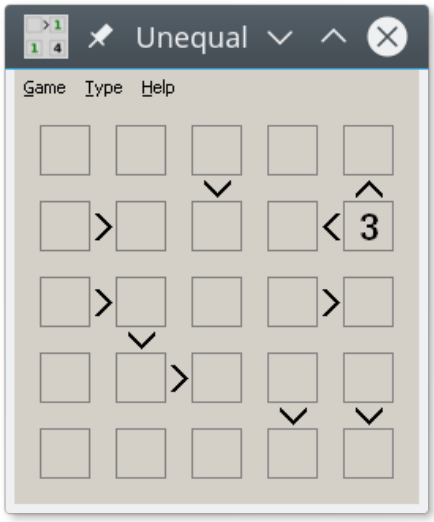
1. Présenter vos programmes de résolution ;

2. Indiquer comment vous générez vos instances ;
3. Indiquer comment vous avez pris en compte la règle difficile de votre jeu le plus difficile (utilisation de callback, ajout de contraintes dès le début de la résolution, ...).
4. Mettre en avant les difficultés rencontrées.

3 Règles

Des règles en anglais plus détaillées peuvent être obtenues sur le site chiark.greenend.org.uk/ sgtatham/-puzzles

3.1 Unequal



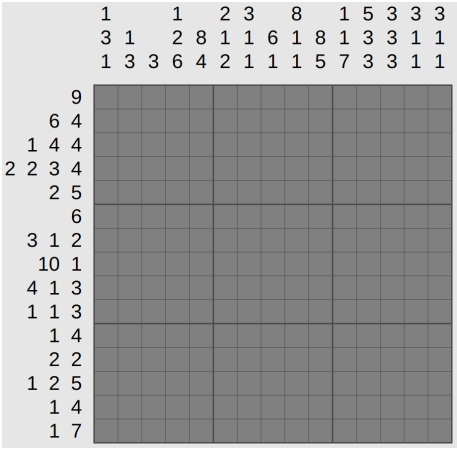
$\Rightarrow$

Exemple de grille initiale

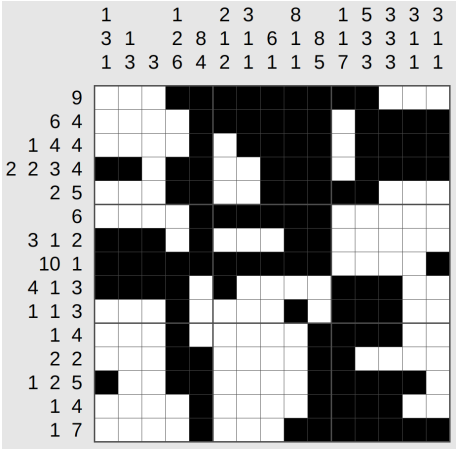
Exemple de grille résolue

— **Règles :** Remplir la grille de taille $n \times n$ par des chiffres de façon à ce que :
  — chaque ligne et colonne contienne les chiffres de 1 à n ;
  — les inégalités soient satisfaites.

3.2 Pattern



$\Rightarrow$

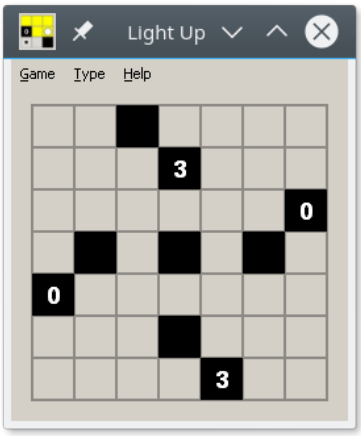


Exemple de grille initiale

Exemple de grille résolue

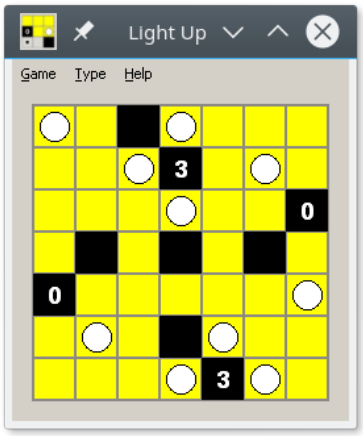
— **Règles :** Remplir la grille de taille $n \times n$ par des cases noires à blanches. Sur une ligne ou une colonne, un ensemble de cases noires consécutives maximal est appelé un *bloc*. Dans chaque ligne et chaque colonne, les nombre indiqués dans la marge représentent le nombre et la taille des blocs à y faire figurer.

3.3 Lightup



Exemple de grille initiale

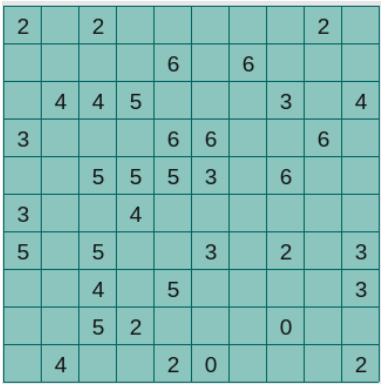
$\Rightarrow$



Exemple de grille résolue

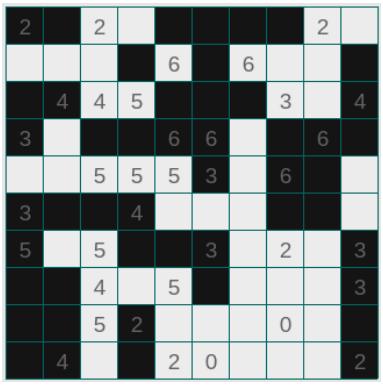
— **Règles :** Placer des lampes afin d'éclairer l'ensemble des cases blanches. Une lampe éclaire toutes les cases blanches figurant sur la même ligne ou la même colonne qu'elle à moins qu'une case noire ne se trouve entre les deux. De plus :
  — Une lampe ne doit pas être éclairée par une autre lampe ;
  — Les cases noires comportant un chiffre x doivent être adjacentes à x lampes.

3.4 Mosaic



Exemple de grille initiale

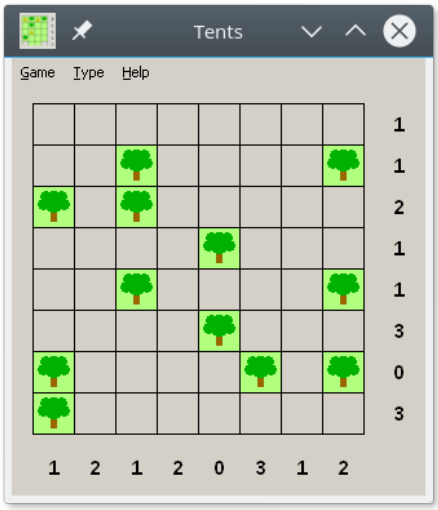
$\Rightarrow$



Exemple de grille résolue

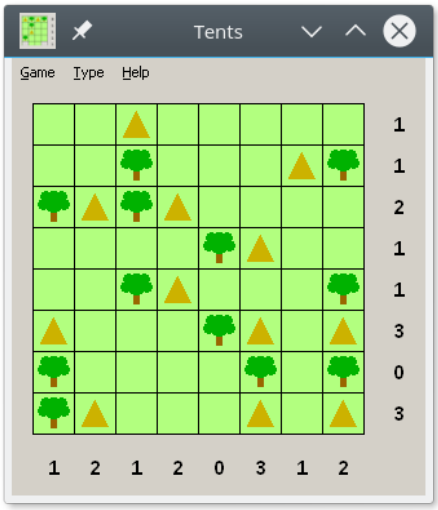
— **Règles :** Placer des cases noires sur la grille de façon à ce que pour chaque nombre $v_{i,j}$ figurant en case (i, j) sur la grille, le nombre de cases noires dans le carré de taille 3x3 centré sur (i, j) soit égal $v_{i,j}$.

3.5 Tents



Exemple de grille initiale

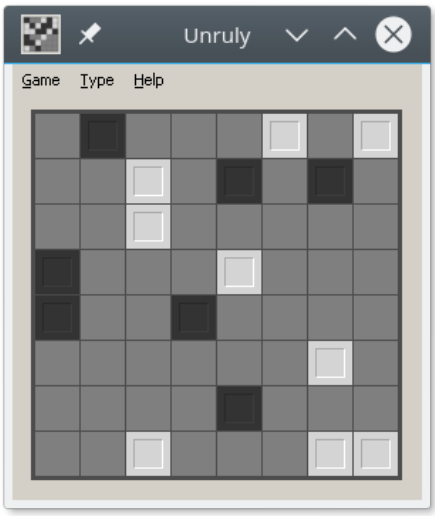
$\Rightarrow$



Exemple de grille résolue

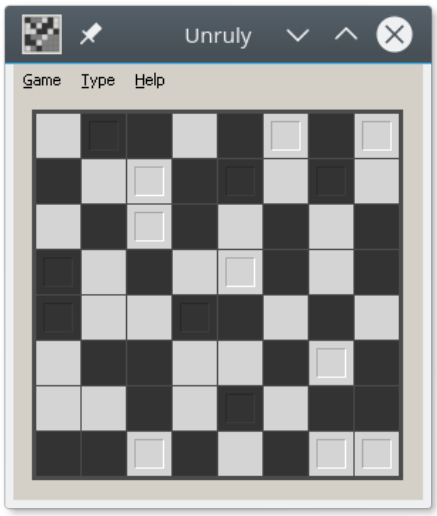
— **Règles :** Placer des tentes de façon à ce que :
  — Chaque ligne et chaque colonne contienne le nombre de tentes indiquées ;
  — Chaque arbre ait une tente adjacente qui lui soit associée (il y a donc autant de tentes que d'arbres, mais un arbre peut, néanmoins, avoir plusieurs tentes qui lui sont adjacentes) ;
  — Deux tentes ne soient jamais adjacentes ou en diagonale l'une de l'autre.

3.6 Unruly



Exemple de grille initiale

$\Rightarrow$



Exemple de grille résolue

— **Règles :** Remplir une grille avec des cases noires et blanches de façon à ce que :
  — Chaque ligne et chaque colonne comporte autant de cases blanches que de noires ; et
  — Il n'y ait jamais plus de 2 cases successives d'une même couleur (horizontalement et verticalement).

3.7 Dominosa

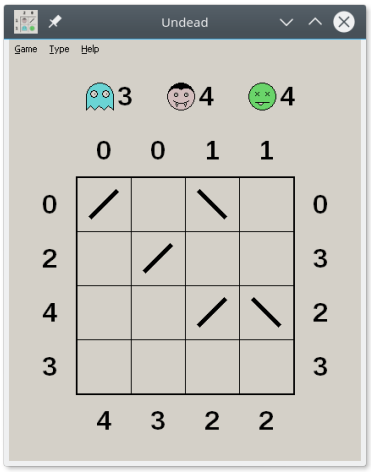
$\Rightarrow$

Exemple de grille initiale

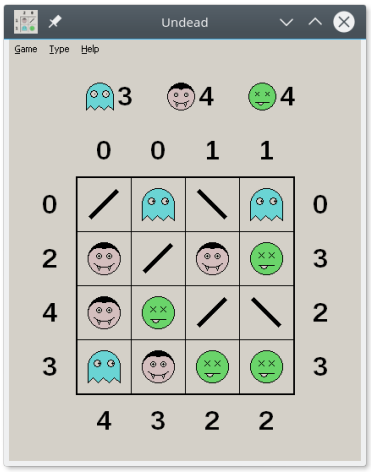
Exemple de grille résolue

— **Règles :** Regrouper par 2 les chiffres afin de former des dominos de façon à ce que tous les dominos soient différents.

3.8 Undead



$\Rightarrow$

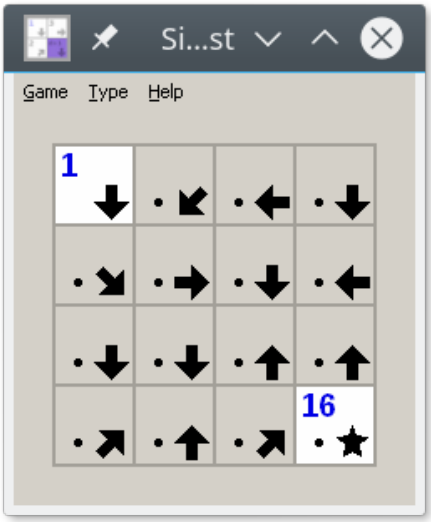


Exemple de grille initiale

Exemple de grille résolue

— **Règles :** Remplir les cases de la grille ne comportant pas de miroirs par des monstres (zombie, vampire ou fantômes). Les chiffres en bord de grille indiquent combien de monstres doivent être visibles par quelqu'un regardant la grille à parir de cette position, sachant que :
  — les zombies sont toujours visibles ;
  — les fantômes ne sont visibles que s'ils sont reflétés ;
  — les vampire ne sont visibles que s'ils ne sont pas reflétés.
  Le nombre de monstres de chaque type à placer est connu.

3.9 Signpost



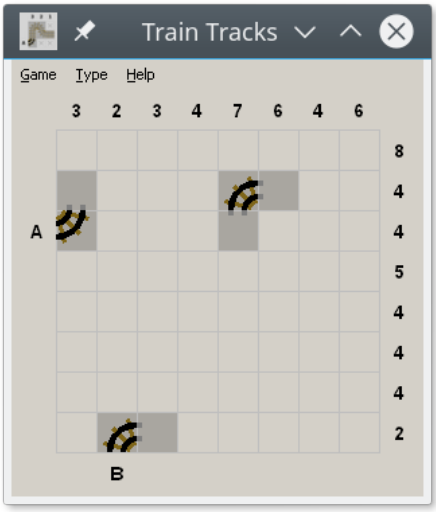
$\Rightarrow$

Exemple de grille initiale

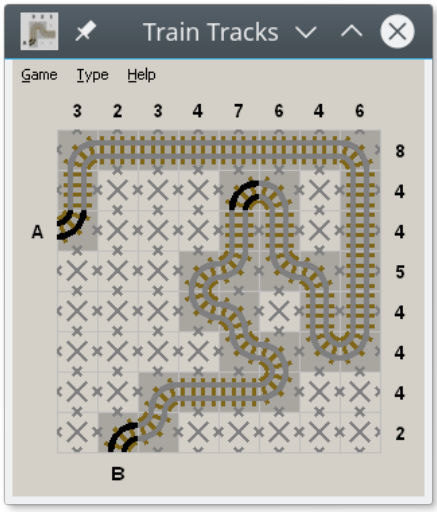
Exemple de grille résolue

— **Règles :** Assigner un numéro à chaque case de façon à ce que le successeur d'une case soit atteignable en suivant la flêche figurant sur la case.

3.10 Tracks



$\Rightarrow$

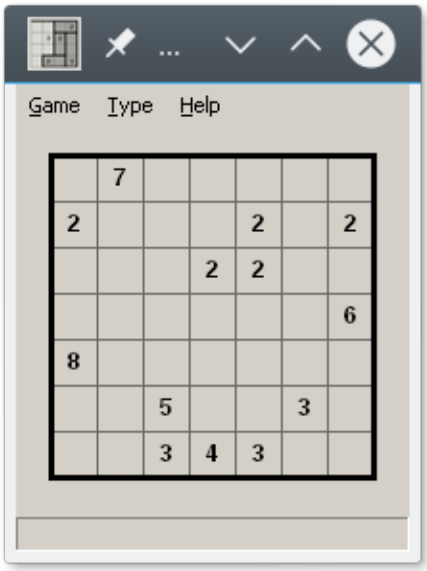


Exemple de grille initiale

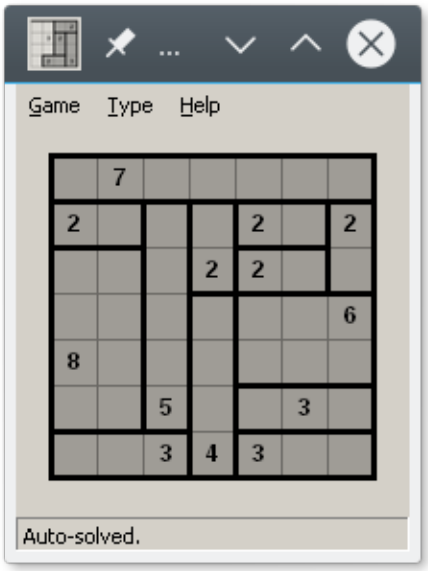
Exemple de grille résolue

— **Règles :** Construire un trajet allant de A en B de façon à ce que le nombre de cases utilisées par le trajet sur chaque ligne et chaque colonne corresponde aux valeurs en bord de grille.

3.11 Rect



$\Rightarrow$

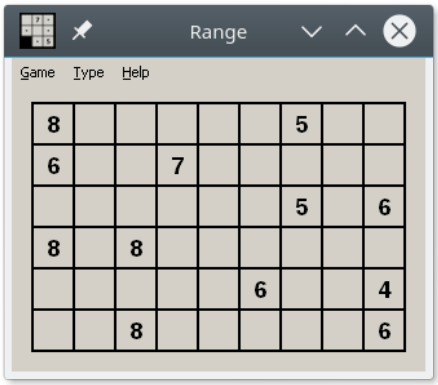


Exemple de grille initiale

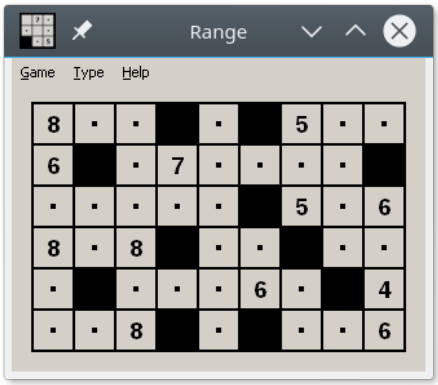
Exemple de grille résolue

— **Règles :** Remplir la grille avec des rectangle de façon à ce que :
  — Chaque chiffre x soit contenu dans un rectangle contenant exactement x cases ;
  — Chaque rectangle contienne exactement un chiffre.

3.12 Range



$\Rightarrow$

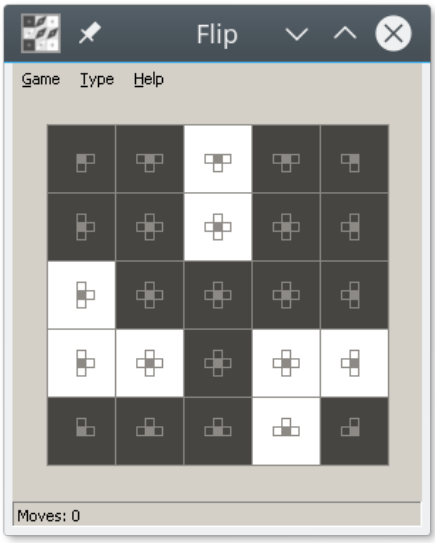


Exemple de grille initiale

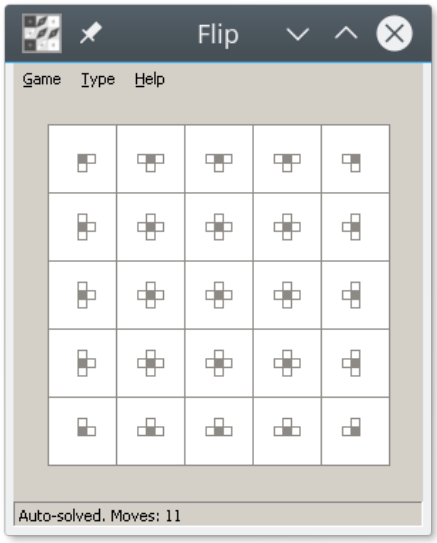
Exemple de grille résolue

— **Règles :** Placer des cases noires de façon à ce que chaque chiffre x puisse *voir* exactement x cases. Un chiffre voit une case si cette case est située sur la même ligne ou la même colonne que lui et si aucune case noire ne figure entre eux deux. De plus, l'ensemble des cases blanches doit former un ensemble connexe (*i.e.,* un ensemble de cases blanches ne peut être isolé des autres par des cases noires) et deux cases noires ne peuvent être adjacentes.

3.13 Flip



$\Rightarrow$

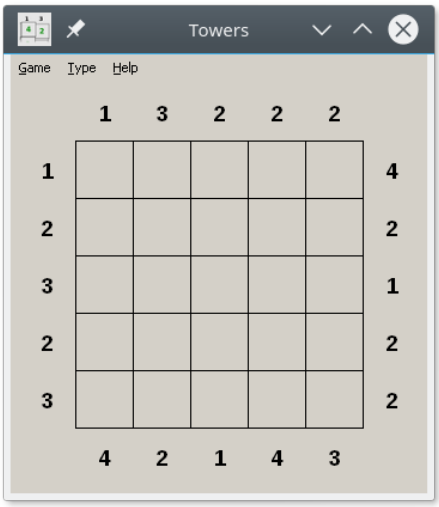


Exemple de grille initiale

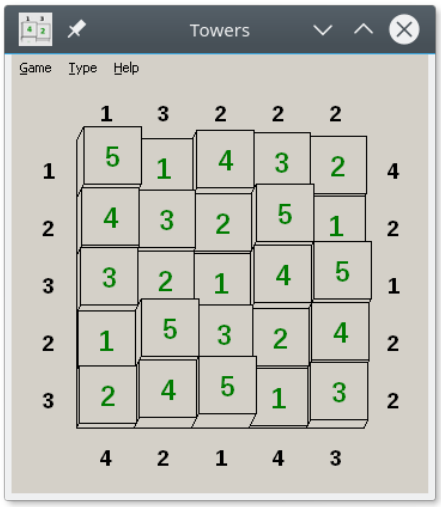
Exemple de grille résolue

— **Règles :** A chaque tour, une case est sélectionnée ce qui change la couleur de cette case et de ses 4 cases voisines. L'objectif est que toutes les cases soient blanches en un nombre de tours.

3.14 Towers



$\Rightarrow$



Exemple de grille initiale

Exemple de grille résolue

— **Règles :** Remplir la grille de taille $n \times n$ par des tours de taille 1 à n de façon à ce que :
  — Chaque ligne et chaque colonne contienne une tour de chaque hauteur ;
  — Les chiffres indiquent combien de tours doivent être visibles par quelqu'un regardant la grille à parir de cette position ;
  Par exemple, si la grille possède un chiffre 1, il n'y a qu'une tour de visible et c'est donc nécessairement une tour de taille n.

3.15 Singles

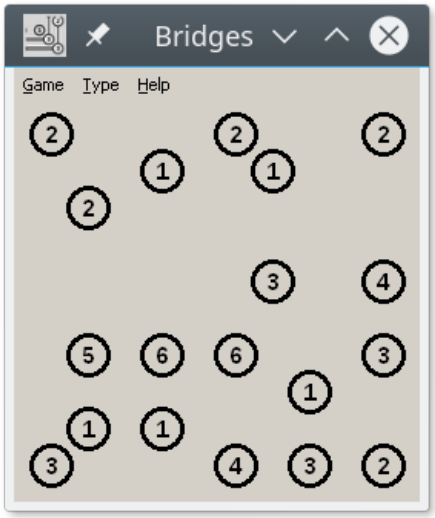
$\Rightarrow$

Exemple de grille initiale

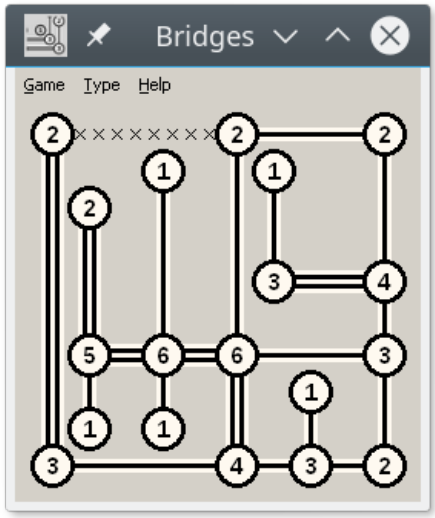
Exemple de grille résolue

— **Règles :** Masquer des case de façon à ce que :
  — aucun chiffre ne soit visible plus d'une fois sur chaque ligne et chaque colonne ;
  — les cases masquées ne soient pas adjacentes (elles peuvent néanmoins être placées en diagonale) ;
  — l'ensemble des cases visibles est connexe.

3.16 Bridges



$\Rightarrow$

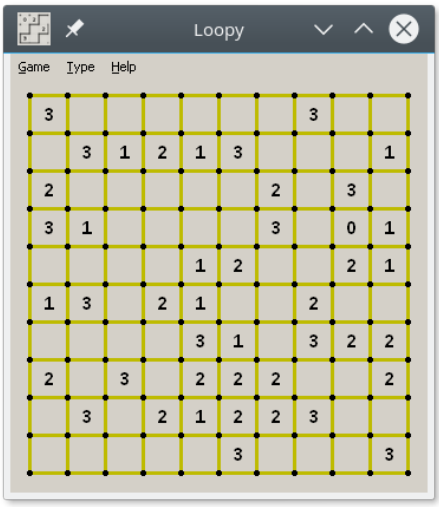


Exemple de grille initiale

Exemple de grille résolue

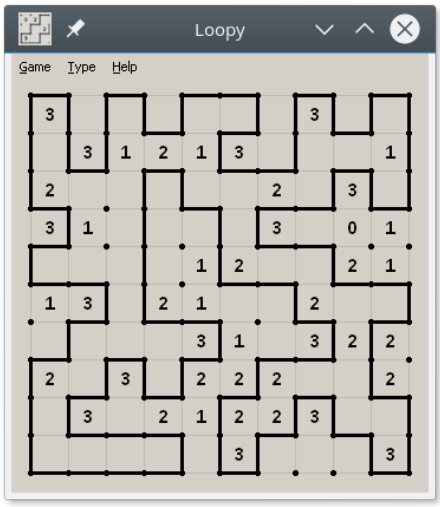
— **Règles :** Ajouter des ponts reliant les sommets de façon à ce que :
  — le nombre de ponts partant d'un sommet soit égal au nombre indiqué sur le sommet ;
  — aucuns ponts ne se croisent ;
  — il existe un chemin entre toutes paires de sommets.

3.17 Loopy



Exemple de grille initiale

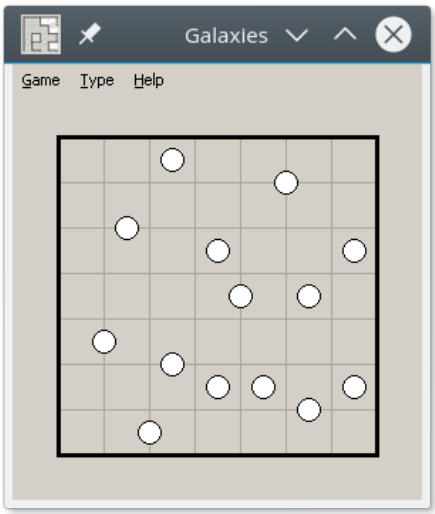
$\Rightarrow$



Exemple de grille résolue

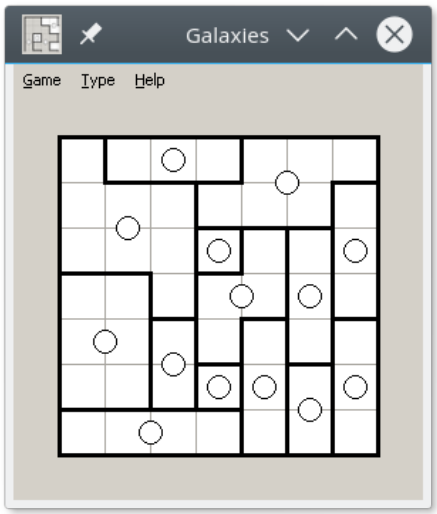
— **Règles :** Tracer une boucle. La boucle doit passer par x cotés d'une case comportant le chiffre x.

3.18 Galaxies



Exemple de grille initiale

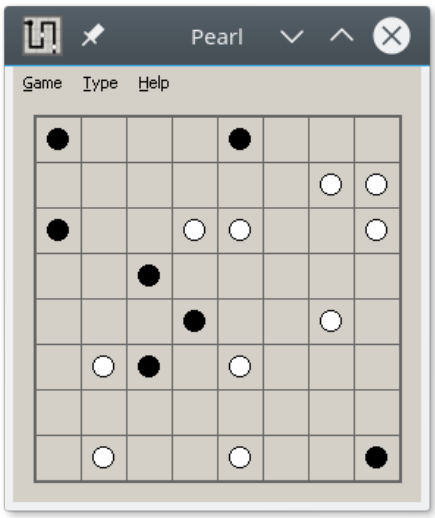
$\Rightarrow$



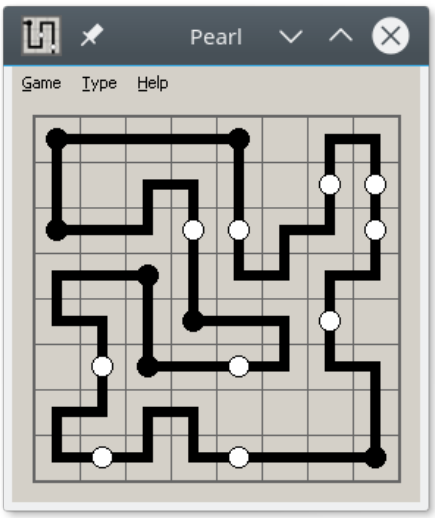
Exemple de grille résolue

— **Règles :** Tracer des séparations afin de diviser la grille en régions. Chaque région doit contenir un unique rond blanc (appelé *galaxie*) de telle sorte que la région soit symétrique par rapport à ce rond.

3.19 Pearl



$\Rightarrow$

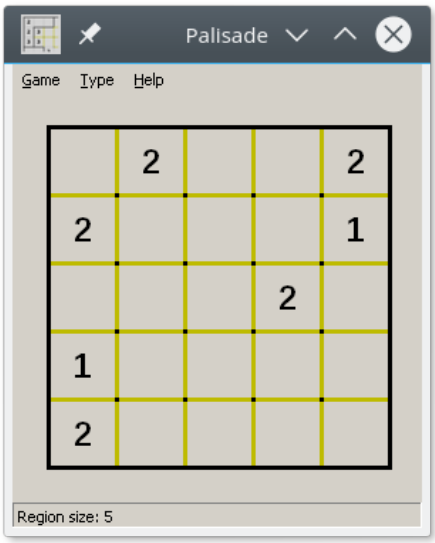


Exemple de grille initiale

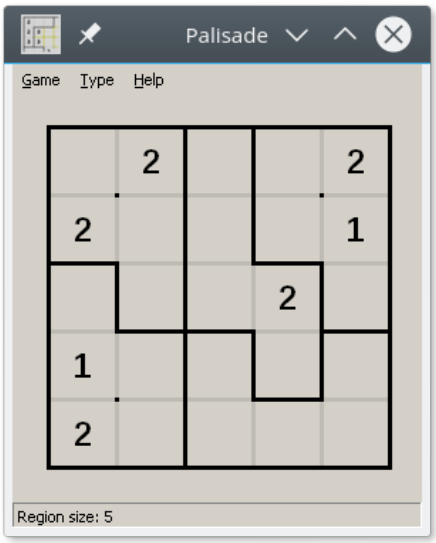
Exemple de grille résolue

— **Règles :** Tracer une boucle de façon à ce que :
  — En chaque case comprenant un rond noir, la boucle forme un angle droit qui ne soit pas directement connecté à un autre angle droit ;
  — En chaque case comprenant un rond blanc, la boucle forme une ligne droite directement connectée à au moins un angle droit.

3.20 Palisade



$\Rightarrow$

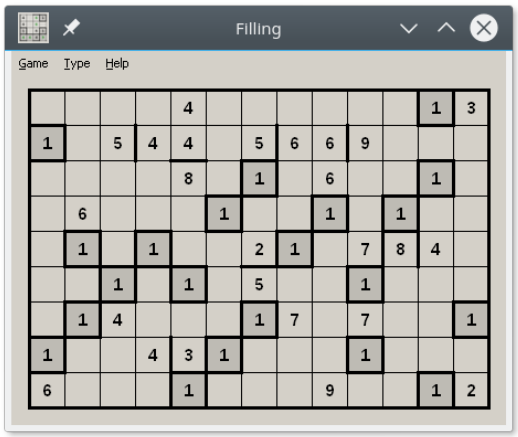


Exemple de grille initiale

Exemple de grille résolue

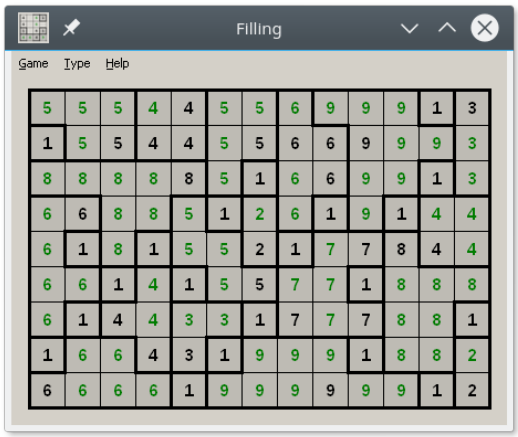
— **Règles :** Placer des palissades afin de former des régions de façon à ce que :
  — Toutes les régions aient la même taille n (la valeur n fait partie des données du problème) ;
  — Chaque case comportant un chiffre x ait exactement x de ses cotés comportant une palissade.

3.21 Filling



Exemple de grille initiale

$\Rightarrow$



Exemple de grille résolue

— **Règles :** Remplir la grille par des chiffres de façon à ce que chaque chiffre x figure dans une zone connexe comprenant x case de valeur x.

4 Utilisation de callbacks

Les jeux de difficulté ≥ 6 contiennent tous une règle qu'il est difficile de modéliser par la programmation linéaire en nombres entiers. Pour prendre en compte ces règles, vous devrez utiliser une fonctionalité de CPLEX appelée *callback*.

Prenons l'exemple d'une variante de sudoku pour illustrer le travail à réaliser. Pour cela on ajoute au sudoku la règle suivante : La distance de Manhattan entre les case (i_1, j_1) et (i_2, j_2) est $|i_1 - i_2| + |j_1 - j_2|$

— Les distances de Manhattan entre la position contenant la valeur 1 dans le block en haut à gauche et les autres positions contenant des 1 doivent toutes être différentes.

La Figure 1 représente une grille invalide et une grille valide pour cette règle.

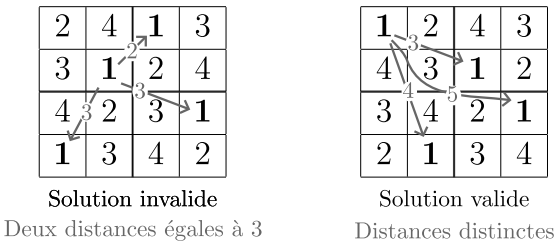


FIGURE 1 – Exemple de solution de grilles de sudoku de taille 4×4. Les distances de Manhattan entre le 1 figurant dans le bloc en haut à gauche et les autres sont représentées.

Il est facile de trouver une contrainte à ajouter au modèle pour rendre infaisable la solution invalide de la Figure 1. En effet, les distances de valeurs 3 étant associées aux cases $(2, 2)$, $(3, 4)$ et $(4, 1)$, on peut interdire cette solution en ajoutant la contrainte : $x_{2,2,1} + x_{3,4,1} + x_{4,1,1} \leq 2$

Cependant, le nombre de solutions qui ne respectent pas la règle étant très élevé (exponentiel en la taille de la grille), on ne peut pas ajouter une contrainte au modèle pour toutes les interdire car :

— Cela augmenterait significativement le temps de calcul des relaxations linéaires à chaque noeud de l'algorithme de branch-and-bound qui deviendrait alors beaucoup plus lent ;

— Pour les instances les plus grandes, cela poserait des problèmes de mémoire.

Pour régler le problème, nous n'allons pas mettre toutes ces contraintes dans le modèle. Au lieu de cela, lorsque CPLEX trouvera une solution, nous testerons si elle vérifie la règle (*i.e.,* si toutes les distances de Manhattan sont bien différentes). Si ce n'est pas le cas, nous ajouterons une contrainte au problème. Pour cela nous utiliserons un callback qui est une fonction que CPLEX appelera à chaque fois qu'il trouve une solution entière.

L'utilisation du callback est présenté dans le fichier `src/resolutionWithCallback.jl`. Voici le code de la fonction callback qui sera appelée par CPLEX :

```
"""
Fonction qui sera exécutée par CPLEX à chaque fois qu'une des conditions
suivantes est remplie :
- 1 solution entière a été trouvée ;
- 1 relaxation a été calculée ;
- ...

Arguments :
- context_id : indique pour quelle raison le callback est appelé ;
- cb_data  :  permet  d'obtenir  d'autres  informations
  (valeur des bornes inférieures et supérieures, meilleure solution
  connue, ...)
"""
function callback_manhattan(cb_data::CPLEX.CallbackContext, context_id::Clong)

    # On teste d'abord si c'est l'obtention d'une solution entière
    # qui a entraîné l'appel du callback
    # (cette fonction isIntegerPoint est définie en fin de fichier mais son
    # contenu n'est pas très important)
    if isIntegerPoint(cb_data, context_id)

        # Cette ligne doit être  appelée avant de pouvoir récupérer la
        # solution entière ayant entraîné l'appel du callback
        CPLEX.load_callback_variable_primal(cb_data, context_id)

        # On récupère la valeur de x dans la solution trouvée par CPLEX
        x_val = callback_value.(cb_data, x)

        # On trouve les coordonnées du 1 dans le carré en haut à gauche
        pTopLeft = nothing
        for i in 1:blockSize
            for j in 1:blockSize
                if x_val[i, j, 1] > 0.9
                    pTopLeft = (i, j)
                end
            end
        end
```

```
        # On trouve  les coordonnées de toutes les cases ayant la valeur 1
        ones_positions = Vector{Tuple{Int,Int}}()
        for i in 1:n, j in 1:n
            if x_val[i, j, 1] > 0.9
                push!(ones_positions, (i, j))
            end
        end

        # Dictionnaire contenant les positions des 1 indexées par
        # leur distance à la position pTopLeft
        # Par exemple si l'index 10 existe, dist_dict[10] contiendra une
        # position p à distance 10 de pTopLeft
        dist_dict = Dict{Int, Tuple{Int,Int}}()

        for a in 1:length(ones_positions)
            p = ones_positions[a]  # On récupère la position

            # Calculer la distance de Manhattan entre p et pTopLeft
            d = abs(pTopLeft[1] - p[1]) + abs(pTopLeft[2] - p[2])

            # S'il existe déjà une autre position à distance d de pTopLeft
            if haskey(dist_dict, d)
                p2 = dist_dict[d] # Récupérer l'autre position

                # Ajouter une contrainte qui empêche d'avoir des 1 dans ces 3
                # positions en même temps
                cstr = @build_constraint(x[pTopLeft[1], pTopLeft[2], 1] +
                                         x[p[1], p[2], 1] +
                                         x[p2[1], p2[2], 1] <= 2)
                MOI.submit(m, MOI.LazyConstraint(cb_data), cstr)
            else
                # Sinon, ajouter au dictionnaire les position à l'indice d
                dist_dict[d] = (p)
            end
        end
    end
end
```

Il reste ensuite à ajouter le callback au modèle pour qu'il puisse l'appeler quand il trouve des solutions.

```
    # Ajout du callback au modèle m
    MOI.set(m, CPLEX.CallbackFunction(), callback_manhattan)

    # Important : Utilisation d'un unique thread
    # (obligatoire quand on utilise un callback)
    MOI.set(m, MOI.NumberOfThreads(), 1)
```

Vous pouvez tester l'utilisation de ce callback en exécutant la fonction `solveDataSetWithCallback()` du fichier `resolutionWithCallback.jl`.

Pour utiliser votre callback dans le jeu difficile il vous faudra :

- Identifier la règle difficile ;
- Ajouter un callback au modèle dans lequel vous devrez :
  - Tester si la solution satisfait la règle (si oui ne rien faire).
  - Si non, ajouter une contrainte qui n'est pas satisfaite par cette solution.